

RL-ALIGN

RL-Kernel

大模型 RL 后训练中的训推一致性实践

v0.1.0

Vensen Mu

RL-Kernel 核心维护者

2026.09.26

github.com/RL-Align/RL-Kernel

分享内容

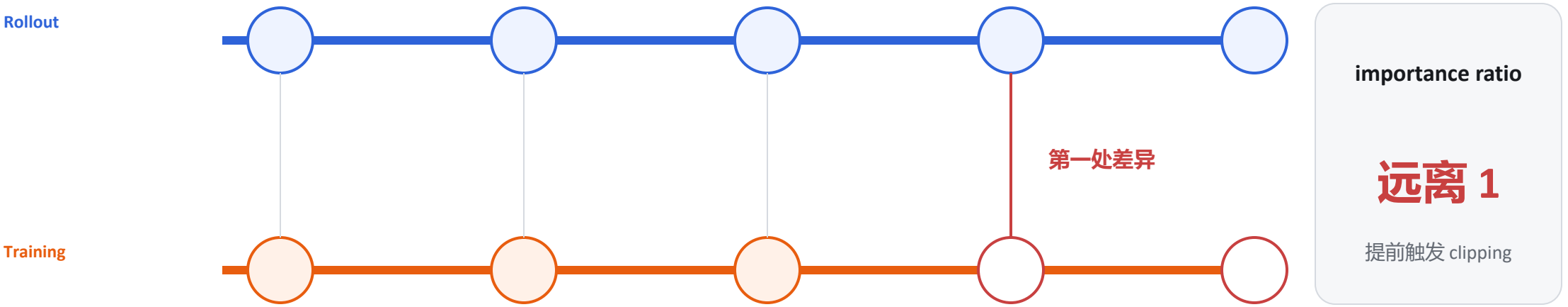
01	**问题背景**	同一模型、同一批 token，为什么两套引擎的 LogP 仍会不同
02	**项目定位**	vime 负责流程，RL-Kernel 对齐关键计算
03	**v0.1.0**	CUDA 和 ROCm 的 200-step 验证结果
04	**当前边界**	已验证范围与下一阶段计划

Rollout LogP 与 Training LogP

阶段	Rollout	Training
执行引擎	**vLLM**	**Megatron-LM**
输入	当前权重 + prompt	同一版权重 + 完整 prompt、 response tokens
计算	生成 response token 并记录 token LogP	对同一批 token 重新计算 selected-token LogP
输出	**rollout_logp**	**train_logp**

比较位置 **rollout_logp** **= ?** **train_logp** **权重更新前**

LogP 偏差会提前改变 importance ratio



权重更新前，train_logp 应与 rollout_logp 一致。但是，由于 train 和 rollout 引擎的底层算子路径不同，两者仍可能出现偏差。出现偏差后，importance ratio 会远离 1，并可能提前触发 clipping。

浮点归约顺序会影响计算结果

同样的四个输入

a　b　c　d

顺序一　((a + b) + c) + d

顺序二　(a + b) + (c + d)

差异来自哪里

GPU会把大规模计算拆成多个局部块。局部结果按不同顺序合并时，浮点舍入发生的位置也不同，所以数学上等价的两条路径，最后几位可能不一样。

常见触发点

GEMM 的 K 维累加　Attention 的 key 归一化
跨 rank 的结果合并　精度转换的位置

LogP 的末位差异会继续影响 importance ratio、KL 和 clipping，RL-Kernel 对齐归约顺序和精度边界。

比较 LogP 前需要对齐五项条件

01

权重版本

同一权重和训练步

02

输入边界

prompt、response、目标 token 和有效 mask

03

位置信息

position、RoPE、causal mask、KV 可见范围。

04

逻辑状态

token / rank 归属、真实词表范围、分片映射。

05

随机条件

seed 与采样状态

完成这五项检查后，才能判断差异是否来自计算路径。vime 固定权重、数据和时序，RL-Kernel 对齐关键计算。

vime 与 RL-Kernel 的分工

vime

负责工作流时序

01 组织 rollout、reference 和 actor train 的先后顺序

02 记录每一轮使用的权重版本和数据流

03 完成采样、训练、更新和权重同步的闭环

RL-Kernel

负责关键计算的数值对齐

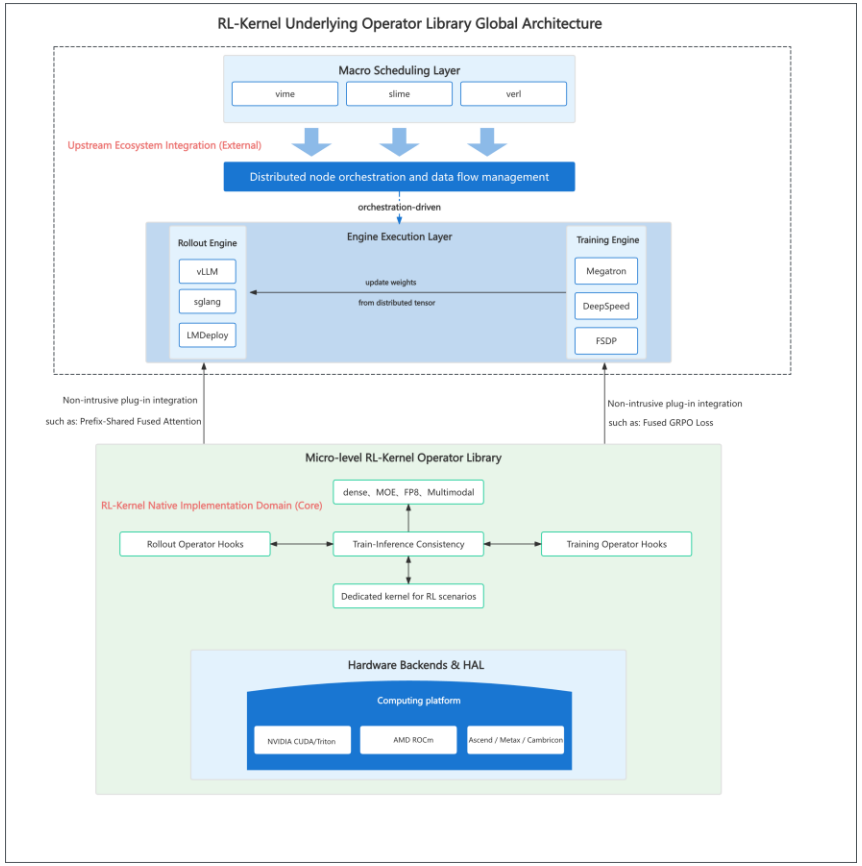
01 算术契约

02 恢复 token 顺序并固定结果合并顺序

03 比较 selected-token LogP 并保存运行记录

vime 保证两边在同一个训练时刻处理同一批数据。RL-Kernel 再保证两边对这批 token 算出相同的 LogP。

RL-Kernel 整体架构



四层关系

01 **调度框架**

vime、slime、verl

02 **执行引擎**

rollout 和 training

03 **一致性算子**

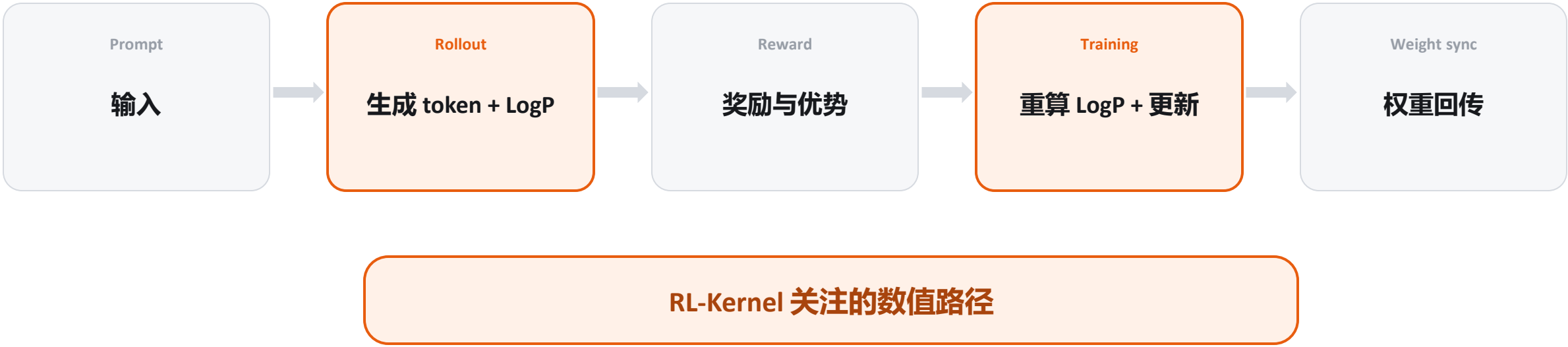
RL-Kernel

04 **硬件后端**

CUDA、ROCm 等

调度框架组织 RL 工作流，rollout 与 training 引擎分别完成生成和训练。RL-Kernel 位于执行引擎与硬件后端之间，提供运行时适配和关键算子。

RL-Kernel 在单个 RL step 中的位置



权重更新前，rollout 记录的 LogP 与 training 重算的 LogP 应对应同一版权重。验证覆盖完整 RL step，同时保留算子级检查。

训推两侧需要对齐的计算

检查项		两套引擎需要保持一致的内容
01	**算术定义**	RMSNorm、 Attention 和 LogP 使用相同的输入范围与归一化定义
02	**Token 顺序**	TP 或 CP 切分后，恢复到同一个逻辑 token 顺序再比较
03	**结果合并**	各个局部块或 rank 的结果按照固定顺序合并
04	**精度边界**	明确哪些步骤用 FP32 累加，以及在哪一步转换精度

vLLM 和 Megatron-LM 可以使用不同的 kernel 和内存布局。对同一批 token，两侧计算出的 selected-token LogP 需要一致。

v0.1.0 覆盖的五类归约

01

RMSNorm

hidden 维归约

02

Attention

可见 key 归约

03

GEMM / SwiGLU

K 维归约

04

Linear LogP

词表归约

05

Collectives

跨 rank 归约

这五类计算都要把局部结果合并为全局结果。切分方式、合并顺序或中间精度发生变化时，差异可能进入最终 LogP。

关键算子的归约维度

模块	主要累加范围	差异来源
RMSNorm	hidden dimension	均值和方差依赖 hidden 维累加顺序
GEMM	K dimension	不同 tile 会改变局部和最终合并方式
Attention	visible keys	softmax 依赖可见 key 的最大值与求和
Linear LogP	vocabulary	LogSumExp 需要在整个词表上归一化
Collectives	ranks	跨卡结果的合并顺序会影响末位结果

排查时先确认参与归约的元素范围，再检查数据切分、结果合并和精度转换位置。

Attention 和 Linear LogP 都依赖 LogSumExp

Attention

对每个 query，在可见 key 上计算 softmax。

局部最大值、指数和累加顺序都会影响归一化结果。

LSE

LogSumExp

Linear LogP

对每个 token，在完整 vocabulary 上计算 log-softmax。

selected-token LogP 依赖同一组 LogSumExp 统计量。

Attention 在可见 key 上归约，Linear LogP 在 vocabulary 上归约。分块和合并顺序会影响最终 LogP。

固定 replay 后定位第一处差异

1 → 2 → 3 → 4

固定 replay

锁定 token、mask 和 position

逐层对比

比较中间结果

找到首个差异

记录对应层

回到在线训练

验证完整 RL step

固定 replay 后，两边处理同一批 token，先排除采样随机性，再逐层定位首个 LogP 差异。

0 mismatch 的验证过程

比较对象、随机条件和实际执行路径都需要固定，并覆盖完整训练过程。

01 对齐权重、token 和 mask

02 固定 seed 和采样参数

03 同一配置重复运行

04 比较 rollout_logp 和 old_logp

05 保存实验配置和比较结果

验证结果

200

完整 RL steps

0

train / rollout mismatch

并行配置命令

README Quick Start

```
./rlk run --tp 4 --rollout-tp 4 --temperature 0.7 --top-p 0.95 \
  --lr 5e-7 --kl-coef 0.01 --max-response-len 6912 \
  --max-tokens-per-gpu 4096 --steps 200
```

公开记录中的并行组合

# Training TP1 CP8, rollout TP1	./rlk run --tp 1 --rollout-tp 1 --temperature 0.7 --top-p 0.95 --steps 200
# Training TP2 CP4, rollout TP4	./rlk run --tp 2 --rollout-tp 4 --temperature 0.7 --top-p 0.95 --steps 200
# Training TP4 CP2, rollout TP8	./rlk run --tp 4 --rollout-tp 8 --temperature 0.7 --top-p 0.95 --steps 200
# Training TP8 CP1, rollout TP2	./rlk run --tp 8 --rollout-tp 2 --temperature 0.7 --top-p 0.95 --steps 200

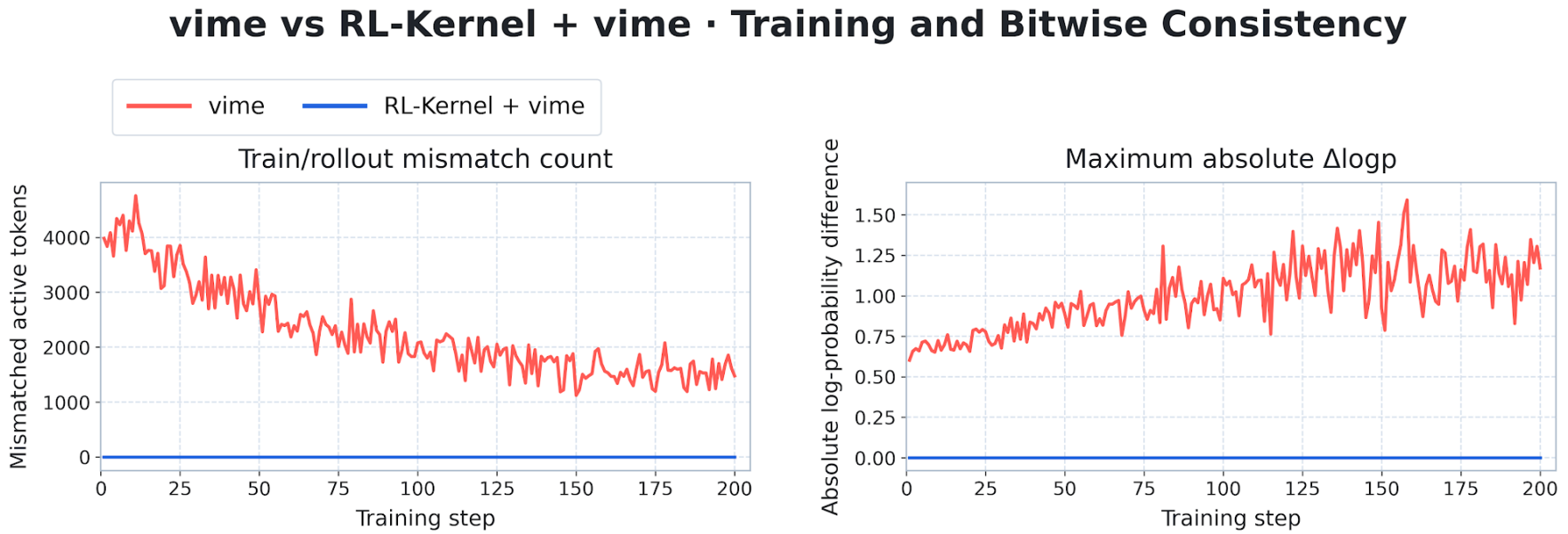
CUDA 实验配置

模型	硬件	并行	全局批次	随机种子
Qwen3-8B	1 node, 8× H100 80GB	TP4 / CP2	128	1234

vime 基线与 RL-Kernel + vime 使用相同模型、数据和训练配置，各运行 200 个 step。后续结果只比较这组配置。

Source: RL-Kernel v0.1.0 Blog / PR #377

CUDA 200 steps： LogP mismatch 为 0



200/200

完整训练 steps

0

RL-Kernel mismatch

红线是 vime 原生路径，蓝线是 RL-Kernel + vime。左图统计不匹配 token 数，右图统计本轮最大的 LogP 绝对差值。蓝线在 200 个 step 中都保持为 0。

Source: RL-Kernel v0.1.0 Blog / PR #377

CUDA 性能不低于原生路径

在保持 0 mismatch 的同时，端到端性能与 vime 原生路径相当。

01	**GEMM**	auto 选择 cuBLASLt no-split-K 或 SM90，SM90 按完整 K 固定顺序累加
02	**RMSNorm**	VIME 严格路径使用 torch RMSNorm，统一归一化语义与参数约束
03	**Attention**	FA4 strict core 关闭 split-K，训练 backward 使用 deterministic 路径
04	**FFN**	packed gate/up、SwiGLU 与 down GEMM 分段执行，复用 packed 权重
05	**LM head / LogP**	训练与 rollout 各自复用 local logits，减少重复 LM-head GEMM
06	**CUDA IPC 通信**	单节点 fixed-tree collective，TP1/2/4/8 使用固定 balanced tree

公开结论仅对应 Qwen3-8B、8×H100 和当前并行配置。

CUDA 快速启动

Qwen3-8B，8×H100。先运行 native，再运行 consistency。

01 安装并指定本地路径

```
python3 -m pip install -e .
export RLK_REPRO_WORKSPACE=/data/rlk-repro
export RLK_REPRO_MODEL_ROOT=/models/Qwen3-8B
```

02 准备数据与 checkpoint

```
rlk-repro prepare \
  --workspace "$RLK_REPRO_WORKSPACE" \
  --download-data --convert-checkpoint
```

03 启动 Ray 并检查环境

```
ray start --head --num-gpus=8 \
  --dashboard-host=127.0.0.1 \
  --object-store-memory=200000000000
rlk-repro doctor --workspace "$RLK_REPRO_WORKSPACE"
```

04 运行基线与一致性路径

```
rlk-repro run --workspace "$RLK_REPRO_WORKSPACE" \
  --mode native --rollouts 8 --wait
rlk-repro run --workspace "$RLK_REPRO_WORKSPACE" \
  --mode consistency --rollouts 8 --wait
```

只需要替换 workspace 与模型路径。标准实验不要求修改 VIME 或 RL-Kernel 脚本。

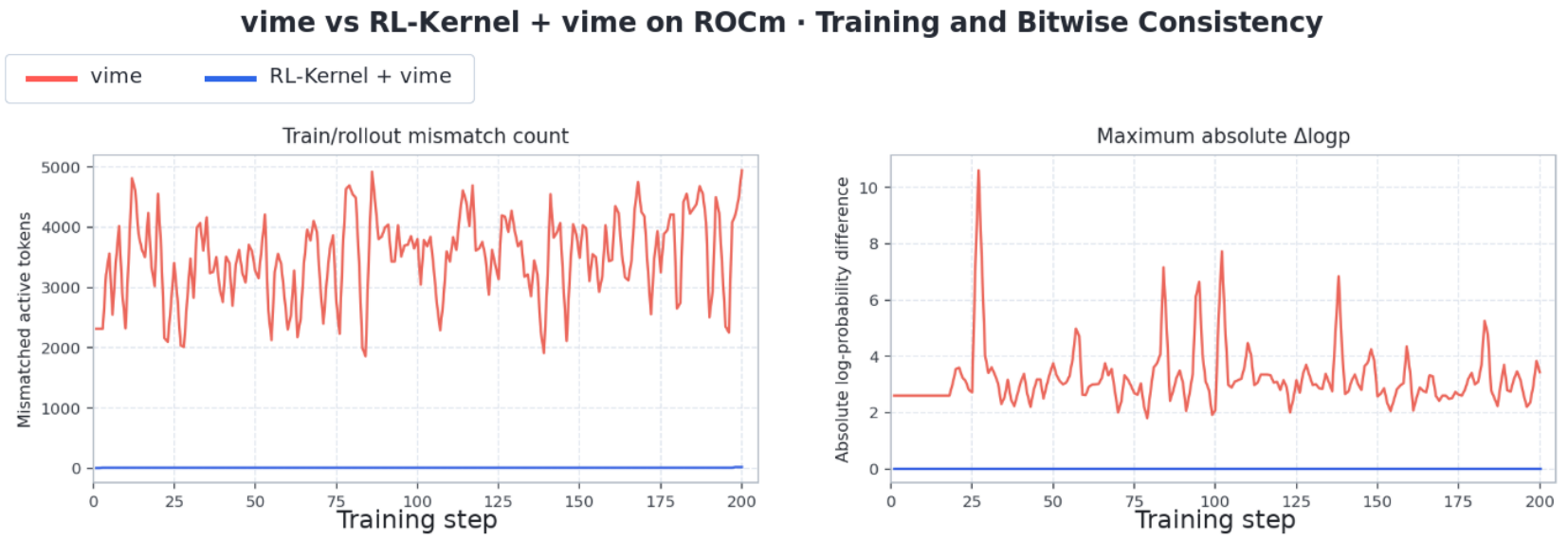
ROCm 实验配置

模型	硬件	并行	全局批次	随机种子
Qwen3-8B	1 node, 8× MI300X 192GB	TP4 / CP2	8	1234

ROCm 侧同样比较 vime 与 RL-Kernel + vime。由于 global batch 与 CUDA 不同，两套硬件的绝对时间不做横向比较。

Source: RL-Kernel v0.1.0 Blog / PR #400

ROCm 200 steps：逐 Bit 一致



200/200

完整训练 steps

0

mismatch

9,400,614

逐元素比较

红线是 vime 原生路径，蓝线是 RL-Kernel + vime。200 个完整 step 共比较 9,400,614 个元素，蓝线在指定比较位置保持完全一致。

Source: RL-Kernel v0.1.0 Blog / PR #400

ROCm 性能与原生路径基本持平

在保持 0 mismatch 的同时，端到端耗时与 vime 原生路径基本持平。

01	**GEMM**	自研 MFMA kernel，固定 chunk 顺序，FP32 合并后写回 BF16
02	**Attention 默认路径**	AITER/CK non-split strict core，paged decode 固定 CK 路径
03	**Attention 可选路径**	Triton chunked attention 固定 block、chunk 与 rescale 合并顺序
04	**FFN**	Triton/MFMA 严格 GEMM，依次执行 packed gate/up、SwiGLU 与 down projection
05	**LogP**	各 TP shard 先算局部统计量，再按固定 vocab tile 顺序合并
06	**ROCm 通信与执行**	HIP IPC 与 RCCL 分层传输，本地固定树归约，配合 HIP Graph 与 cache 复用

公开结论仅对应 Qwen3-8B、8×MI300X 和当前并行配置。

ROCm MI300X 快速启动

机器差异写入 profile，本次实验参数直接由 CLI 覆盖

01 选择公开的机器 profile

```
cd /path/to/RL-Kernel
export PATH="$PWD/bin:/opt/venv/bin:$PATH"
export RLK_REPRO_PROFILE="$PWD/examples/
  vime_rocm_attention_ablation/profiles/mi300x-qwen3-8b.json"
```

02 执行一轮公开配置

```
./rlk run --tp 2 --rollout-tp 4 \
  --temperature 0.7 --top-p 0.95 --steps 1
```

03 只用 CLI 改拓扑和采样

```
./rlk run --tp 8 --rollout-tp 4 \
  --temperature 1.3 --top-p 0.8 --steps 1
```

04 检查 200-step 启动命令

```
./rlk plan --tp 4 --rollout-tp 2 \
  --temperature 1 --top-p 1
```

profile 保存机器路径和默认配置。显式 CLI 参数覆盖 profile。plan 只检查最终命令，不启动训练。

v0.1.0 做了什么，还有哪些待做

已完成验证

200 个训练 step：LogP mismatch 为 0

CUDA 与 ROCm：公开曲线和运行记录

vLLM 与 Megatron-LM：selected-token LogP 对齐

Qwen3-8B Dense：提供单机快速启动

仍需验证

消除 LogP mismatch 后，训练收益是否稳定提升

MoE、多模态和多机

下一阶段接入计划

01 当前路径

Dense

CUDA / ROCm
已完成公开验证

02

DeepSeek-V4

Flash MoE 开发中

03

Gemma / Qwen3-Next

模型与算子 RFC

04

更多框架

verl / AReaL 接入

05

更多后端

MUSA / Ascend 继续完善

四项工作并行推进，每个方向都需要单独完成配置、关键计算检查和端到端训练测试。

Q&A

Q&A

欢迎提问

项目定位、验证方法、CUDA / ROCm 结果、下一步计划

github.com/RL-Align/RL-Kernel